

5.DIVIDE AND CONQUER METHOD:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
struct Point {
```

```
    int x, y;
```

```
};
```

```
float dist(struct Point p1, struct Point p2) {
```

```
    return sqrt((p1.x - p2.x) * (p1.x - p2.x) +
```

```
                (p1.y - p2.y) * (p1.y - p2.y));
```

```
}
```

```
int compareX(const void *a, const void *b) {
```

```
    return ((struct Point *)a)->x - ((struct Point *)b)->x;
```

```
}
```

```
int compareY(const void *a, const void *b) {
```

```
    return ((struct Point *)a)->y - ((struct Point *)b)->y;
```

```
}
```

```
float bruteForce(struct Point P[], int n) {
```

```
    float min = FLT_MAX;
```

```

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (dist(P[i], P[j]) < min)
                min = dist(P[i], P[j]);

    return min;
}

```

```

float stripClosest(struct Point strip[], int size, float d) {
    float min = d;

    qsort(strip, size, sizeof(struct Point), compareY);

    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size &&
            (strip[j].y - strip[i].y) < min; j++) {
            float distance = dist(strip[i], strip[j]);
            if (distance < min)
                min = distance;
        }
    }

    return min;
}

```

```

float closestUtil(struct Point P[], int n) {
    if (n <= 3)
        return bruteForce(P, n);
}

```

```

int mid = n / 2;

struct Point midPoint = P[mid];

float dl = closestUtil(P, mid);
float dr = closestUtil(P + mid, n - mid);

float d = (dl < dr) ? dl : dr;

struct Point strip[n];
int j = 0;

for (int i = 0; i < n; i++)
    if (abs(P[i].x - midPoint.x) < d)
        strip[j++] = P[i];

float stripDist = stripClosest(strip, j, d);

return (d < stripDist) ? d : stripDist;
}

float closest(struct Point P[], int n) {
    qsort(P, n, sizeof(struct Point), compareX);
    return closestUtil(P, n);
}

int main() {
    int n;

```

```

printf("Enter number of points: ");

scanf("%d", &n);

struct Point P[n];

printf("Enter the coordinates (x y):\n");

for (int i = 0; i < n; i++)

    scanf("%d %d", &P[i].x, &P[i].y);

printf("Minimum Distance = %.2f\n", closest(P, n));

return 0;
}

```

OUTPUT:

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <float.h>
5
6 struct Point {
7     int x, y;
8 };
9
10
11 float dist(struct Point p1, struct Point p2) {
12     return sqrt((p1.x - p2.x) * (p1.x - p2.x) +
13                (p1.y - p2.y) * (p1.y - p2.y));
14 }
15
16
17 int compareX(const void *a, const void *b) {
18     return ((struct Point *)a)->x - ((struct Point *)b)->x;
19 }
20
21
22 int compareY(const void *a, const void *b) {

```

Output

```

Enter number of points: 6
Enter the coordinates (x y):
1 2
1 4
3 5
3 4
5 6
7 8
Minimum Distance = 1.00

=== Code Execution Successful ===

```